iscte TECNOLOGIAS E ARQUITETURA

Software Engineering

Code Smells

Outline

- Motivation
- Martin Fowler’s catalog of code smells
- Tools for code smells detection

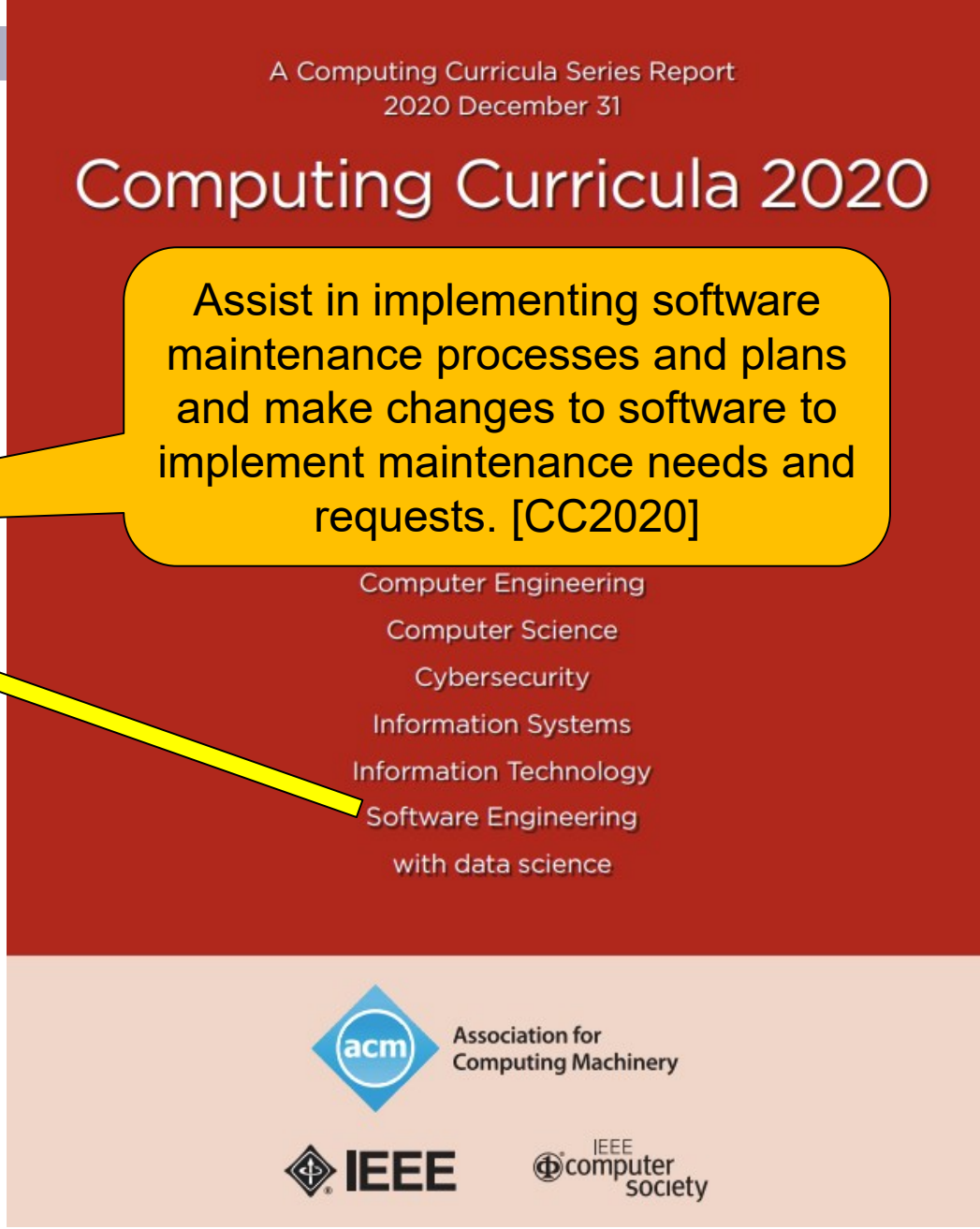
Soft. Eng. Competences

- Software Requirements
- Software Design
- Software Construction
- Software Testing
- Software Sustainment
- Software Process and Life Cycle
- Software Systems Engineering
- Software Quality
- Software Security
- Software Safety
- Software Configuration Manag.
- Software Measurement
- Human-Computer Interaction
- Project Management
- Behavioral Attributes

CDSI

IPM

Assist in implementing software maintenance processes and plans and make changes to software to implement maintenance needs and requests. [CC2020]







1. Software Requirements
2. Software Architecture
3. Software Design
4. Software Construction
5. Software Testing
6. Software Engineering Operations
7. Software Maintenance
8. Software Configuration Management
9. Software Engineering Management
10. Software Engineering Models and Methods
11. Software Engineering Process
12. Software Quality
13. Software Security
14. Software Engineering Economics
15. Software Engineering Professional Practice
16. Computing Foundations
17. Mathematical Foundations
18. Engineering Foundations

Generally, software visualization tools are accompanied by various quality assurance features, such as quality metrics calculation, technical debt estimation, and bad design and coding practices (code smells) detection. [SWEBOK 4.0]

When something smells bad, there might be a problem



“Smelly” is not always a problem

– A smell could be the indication of a problem



– But sometimes there might not be any problem at all



Code smells

- **Code smells** refer to symptoms of reduced understandability and/or other maintainability problems
  - Can occur both in source code and test code

According to Martin Fowler:

"*a **code smell** is a surface indication that usually corresponds to a deeper problem in the system*"

Example

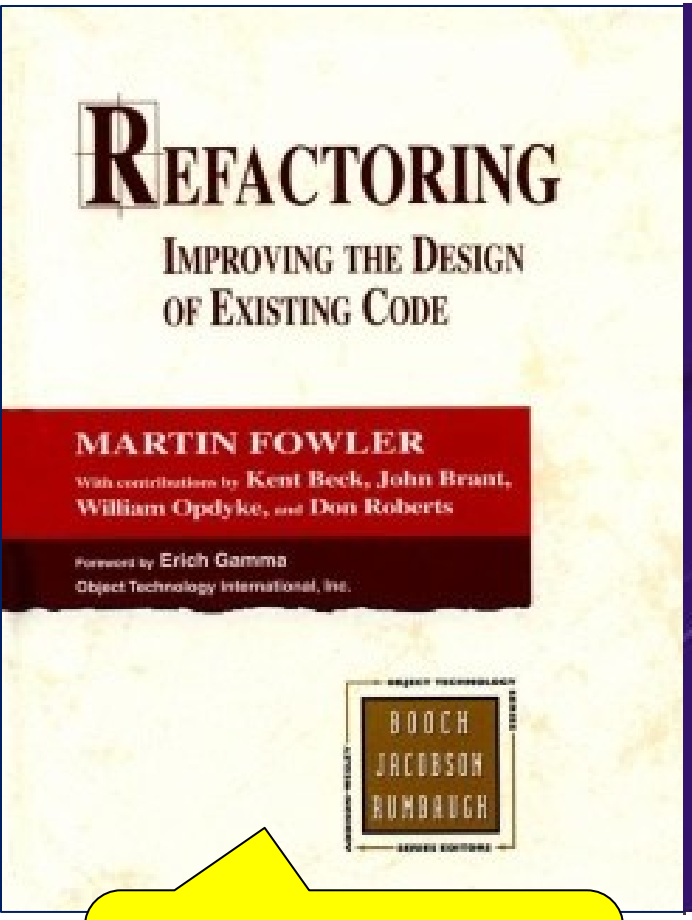
A code smell such as *Large Method* is often indicative of a problem, but sometimes long methods are not a problem.

For instance, when setting up a GUI, reading well-defined file formats (JPEG, GIF), etc. large methods may be the best alternative.

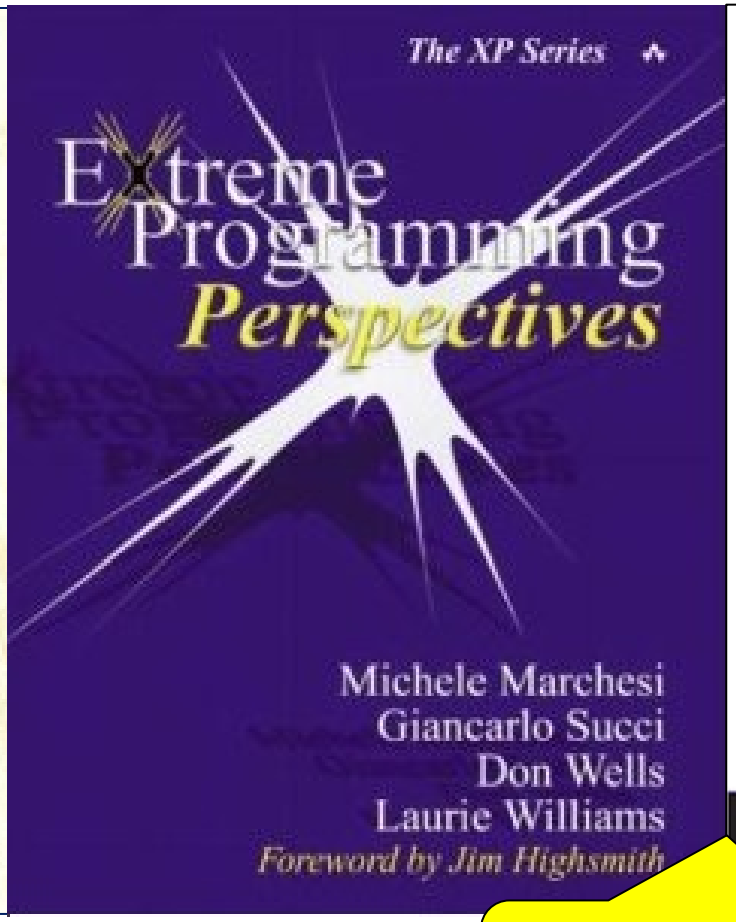
Detecting code smells

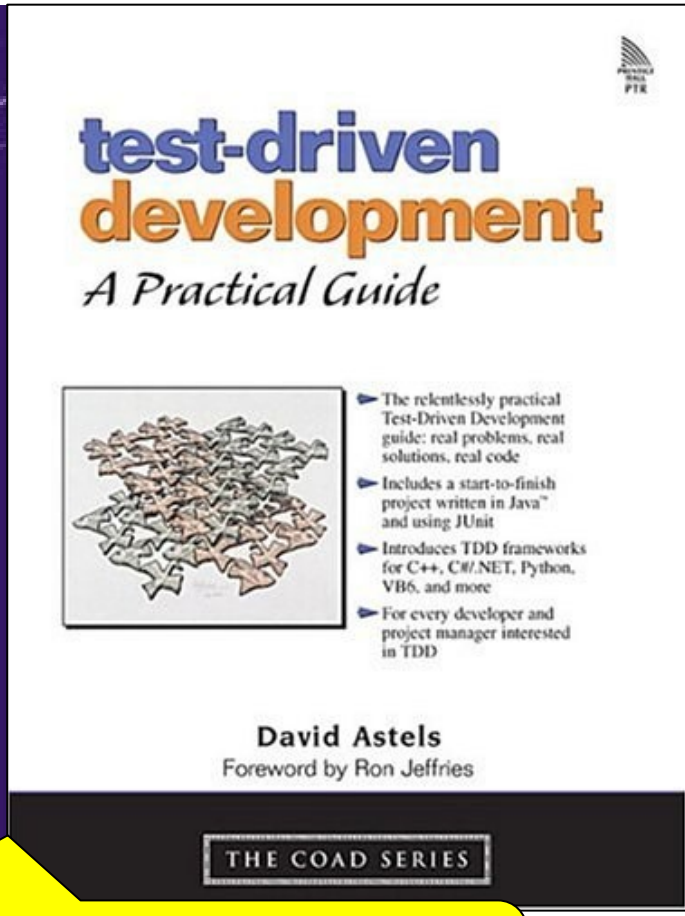
- To detect something, we must know what we are talking about
- **Good news:** Code smells have been catalogued
- **Bad news:** most tools only detect a few of those catalogued code smells

Code smells catalogs



This book proposes a source code smells catalog





These two books propose test code smells catalogs

Example: Detection with JDeodorant



Java - VideoStore_Refactoring/src/v0/Customer.java - Eclipse

File Edit Source Refactor Navigate Search Project Bad Smells Run Window Help

God Class
Long Method
Type Checking
Feature Envy

Package Explorer | Type Hierarchy

use-3.0.6
VideoStore_Refactoring
src
v0
Customer.java
Main.java
Movie.java
Rental.java
v0.mgc
v0.png
v1
v2
v3
v4
v5
v6
v7
v8
v9
vNew
vPersistenceDemo
tests
v0
Test_v0.java
v9
Test_v9.java
vNew
Test_vNew.java

Main.java | Main.java | Customer.java | Customer.java

src ▸ v0 ▸ Customer ▸ statement() : String

```
35  double thisAmount = 0;
36
37  // determine amounts for each line
38  switch (each.getMovie().getPriceCode())
39  {
40    case REGULAR:
41      thisAmount += 2;
42      if (each.getDaysRented() > 2)
43        thisAmount += (each.getDaysRented()
44      break;
45    case NEW_RELEASE:
46      thisAmount += each.getDaysRented() * 3;
47      break;
48    case CHILDRENS:
49      thisAmount += 1.5;
50      if (each.getDaysRented() > 3)
```

JDeodorant allows to detect (only) 4 code smells and apply adequate refactorings to eliminate them

Problems | Search | Console | God Class | Long Method | Feature Envy | Type Checking

Refactoring Type	Source Method	Variable Criterion
	v0.Customer::public java.lang.String statement()	thisAmount
Extract Method		
	v0.Main::public static void main(java.lang.String[])	who
	v0.Customer::public java.lang.String statement()	frequentRenterPoints
	v0.Customer::public java.lang.String statement()	totalAmount

Removing code smells

- **Why?**
  - Removing code smells may improve source code maintainability and extensibility
- **How?**
  - Analyzing each code smell instance and determine if it is really a problem
  - Applying appropriate refactorings

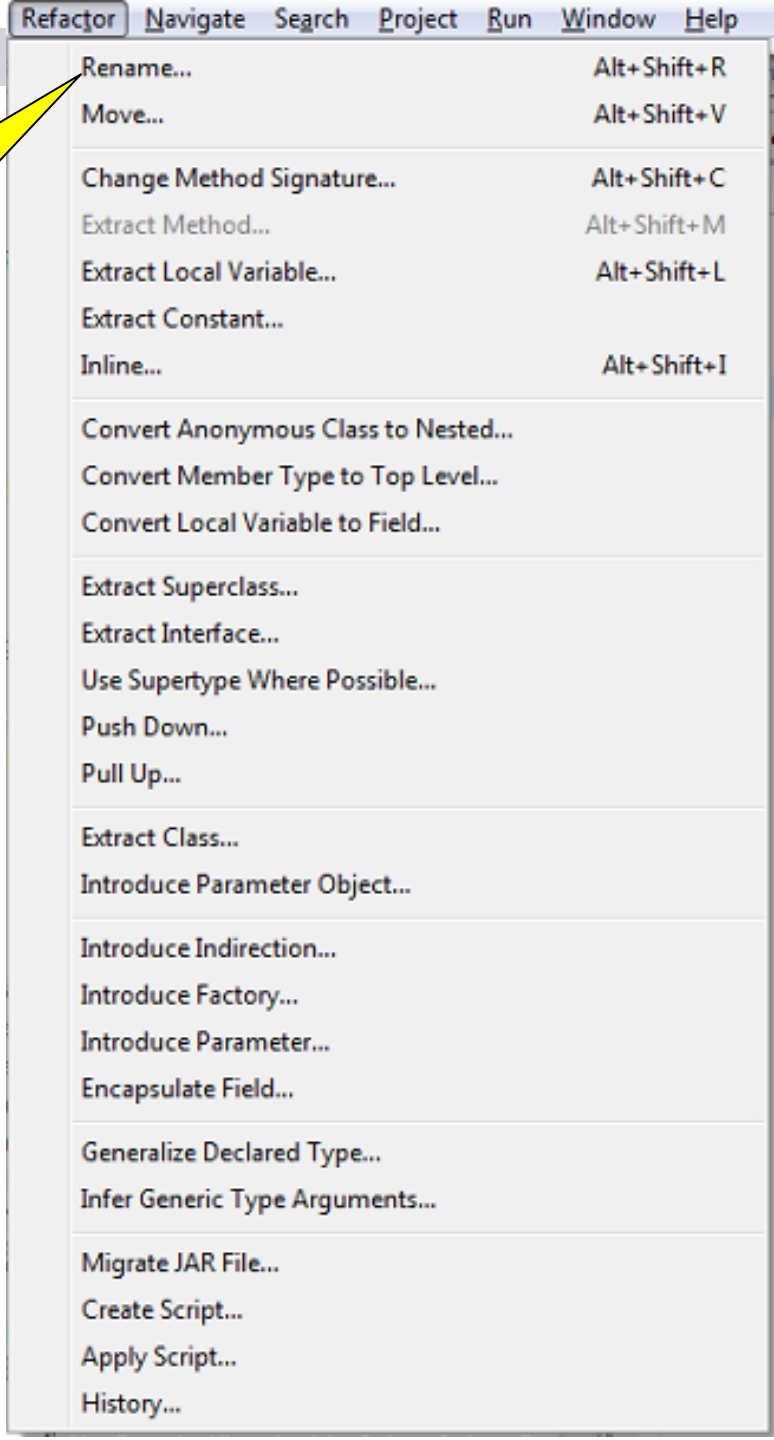
Code refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior

Refactoring in Eclipse

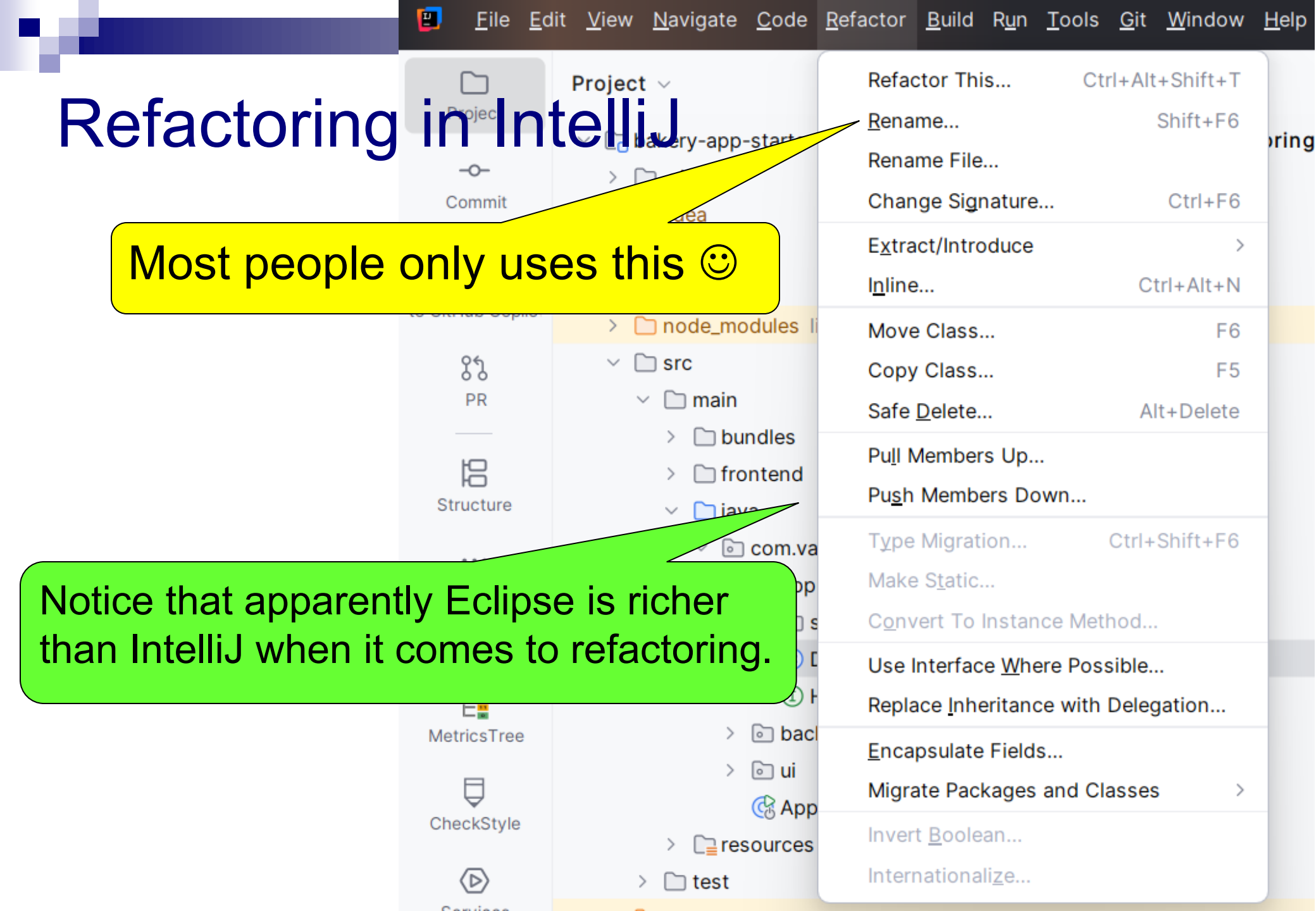
Most people only uses this ☺

For a better comprehension on the relationship between code smells and refactoring operations see this playlist after reading the current slides





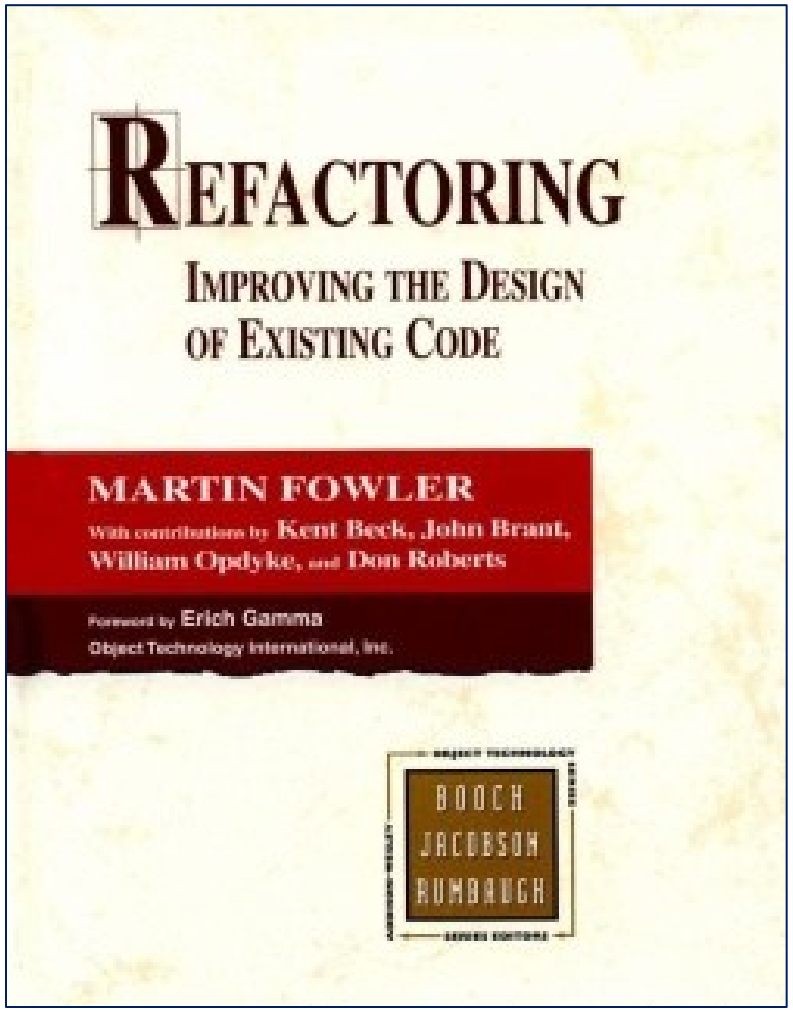
Refactoring in IntelliJ



Most people only uses this ☺

Notice that apparently Eclipse is richer than IntelliJ when it comes to refactoring.

Source Code Smells (Martin Fowler)

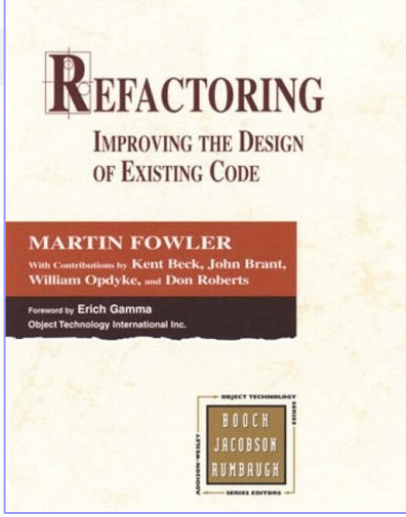


ADDITIONAL READING:

Chapter: "Bad Smells in code" (pp 63-72)

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

Martin Fowler's Catalog



- Alternative Classes with Different Interfaces
- Comments
- Data Class
- Data Clumps
- Divergent Change
- Duplicated Code
- Feature Envy
- Inappropriate Intimacy
- Incomplete Library Class
- Large Class
- Lazy Class
- Long Method
- Long Parameter List
- Message Chains
- Middle Man
- Parallel Inheritance Hierarchies
- Primitive Obsession
- Refused Bequest
- Shotgun Surgery
- Speculative Generality
- Switch Statements
- Temporary Field



Code smells drivers

Five software properties capture both its **structural quality** (coupling, cohesion, complexity) and the **semantic clarity** (readability, expressiveness) that drive the emergence — or absence — of code smells.

Coupling

- The degree of **interdependence between modules or classes**. High coupling means that a change in one part of the system tends to affect many others, making the code harder to modify, test, and reuse.
  → *Goal:* Keep modules as independent as possible.

Code smells drivers

Cohesion

- The degree to which the **elements inside a single module or class work together** toward one, well-defined purpose.
  Low cohesion means a class or function handles unrelated responsibilities, reducing clarity and reusability.
  → *Goal:* Each unit should do one thing and do it well.

Complexity

- A measure of how **complicated the internal logic or structure** of code is — including control flow, nesting, and dependencies.
  High complexity increases the cognitive load for understanding, testing, and maintaining the code.
  → *Goal:* Keep logic simple and straightforward.

Code smells drivers

Readability

- The degree to which code is **easy to understand by humans** — influenced by naming, formatting, clarity of intent, and consistency. Low readability leads to misunderstanding and errors even when the underlying design is sound.
  → *Goal:* Make code communicate its purpose clearly.

Expressiveness

- The degree to which the code **accurately and clearly models the problem domain**, using appropriate abstractions and representations.
  Low expressiveness (e.g., overuse of primitives, poor abstractions) makes intent obscure and invites duplication or misuse.
  → *Goal:* Represent domain concepts explicitly and meaningfully.

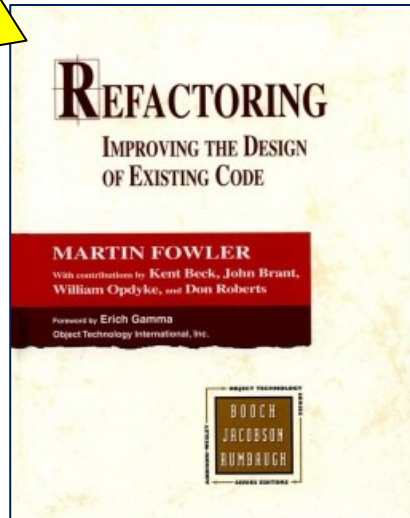
Code smells by intent / driver

Intent	Typical Smells	Main Driver	Design Problem
Over-complex structure	Long Method, Large Class, Duplicate Code	Complexity	Code grows too large or too intricate to reason about.
Low cohesion / mixed responsibilities	Divergent Change, Temporary Field, Lazy Class, Data Clumps	Cohesion	One class handles too many unrelated roles.
Tight coupling / dependency tangle	Shotgun Surgery, Feature Envy, Inappropriate Intimacy, Message Chains, Parallel Hierarchies	Coupling	Changes ripple through the system; hard to modify independently.
Poor abstraction / weak domain modeling	Primitive Obsession, Data Class, Speculative Generality, Alternative Classes w/ Diff. Interfaces, Switch Statements	Expressiveness	Code does not reflect domain concepts clearly; missing or misused abstraction.
Readability / clarity defects	Comments, Dead Code, Long Parameter List	Readability	Code is harder to understand due to surface-level presentation issues.

Long Method

- It is not only about method size, as measured by LOC
- It is also about complexity, namely the profusion of conditional statements and loops, exception handling, …

- *Longer procedures are more difficult to understand*
- *Whenever we feel the need to comment something, we write a method instead*
- *The key is not method length but the semantic distance between what the method does and how it does it*
- *Look for comments to find the code to extract This often signals this semantic distance*
- *Conditionals and loops also give signs for extractions*



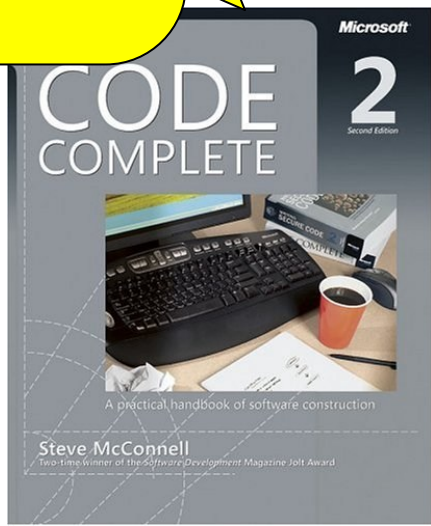
Long Method

- Criteria for deciding if we have a long method are a bit fuzzy / subjective

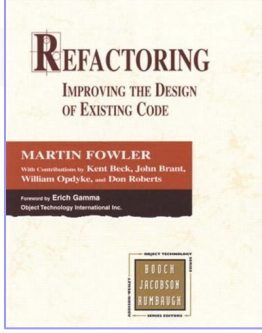
- *Accessor routines should be very short*
- *Issues such as depth of nesting, number of variables, and other complexity related considerations should dictate the length of the routine, rather than imposing a length restriction per se*
- *A complex algorithm can grow up to 100 – 200 LOC*
- *Routines longer than 200 LOC decrease understandability*

WARNING:

The inherent subjectivity in some code smells definition makes it more easier to assess the existence of code smells in a relative fashion, rather than on a absolute (final and decisive) way



Feature Envy

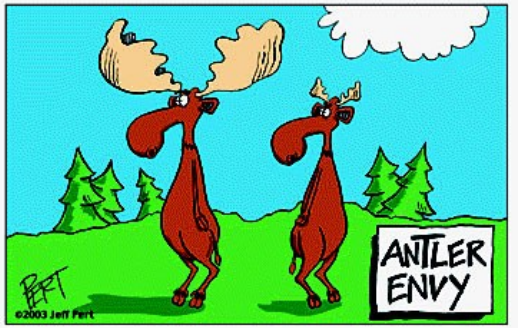


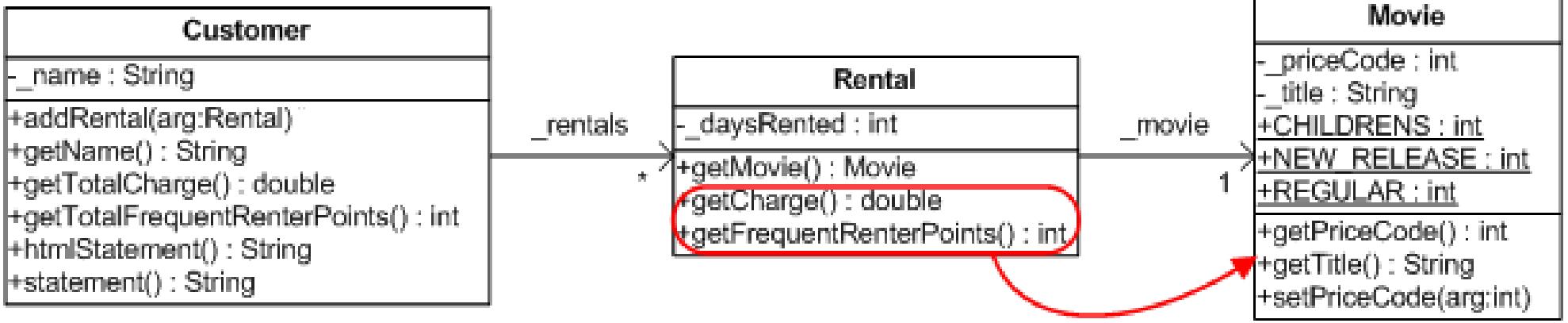
A method making more use of the public features of another class than it does of its own

RATIONALE / ADVICE

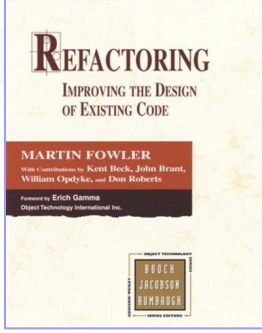
- The whole point of objects is that they are a technique to package data with the processes used on that data When a method seems more interested in a class other than the one it actually is in, this is “feature envy”!

Feature Envy





REFACTORING EXAMPLE:
If the method clearly wants to be elsewhere, you use Move Method to get it there. Sometimes only part of the method suffers from envy; in that case use Extract Method on the jealous bit and Move Method to give it a dream home



Inappropriate Intimacy

Classes using too many things that should be private in other classes

RATIONALE / ADVICE

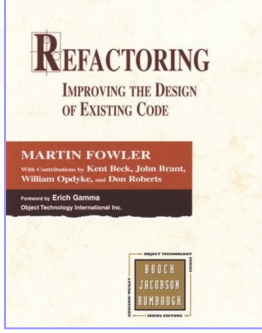
- This smells compromises the other class's encapsulation
- Classes should know as little as possible about each other



In Java this can be avoided with the **private** visibility modifier. What about the other three?

Refused Bequest

A class does not use things it inherits from its superclass (e.g., by overriding methods



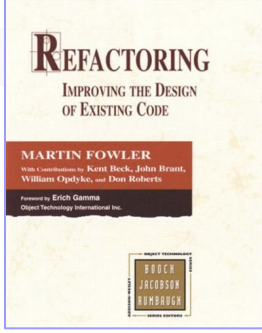
RATIONALE / ADVICE

- This smell results from inheriting code you do not want
- Unwanted bequest results in name space pollution

Name space pollution can be avoided by making features (attributes and methods) **private**. To avoid a class from being subclassed (therefore blocking bequests) use the **final** modifier. Many of the Java standard library classes are final, such as java.lang.System and java.lang.String.

Declaring methods as "final" prevents overriding. This enforces consistent behavior and prevents subclasses from refusing inherited responsibilities.





Duplicated Code (aka code clones)

Code is repeated in multiple places

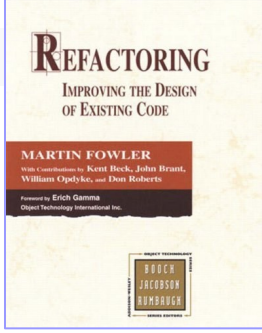
RATIONALE / ADVICE

- This is the most pervasive and pungent code smell
  - "clones", are due to cut & paste practices
- Clones in excess imply that maintenance effort is multiplied, since all code must be maintained
- Generality (polymorphism) techniques should be used to mitigate this problem



Generality is achieved in Java through:

- *Inheritance polymorphism*
- *Interface polymorphism*
- *Parametric polymorphism (generics)*



Speculative Generality

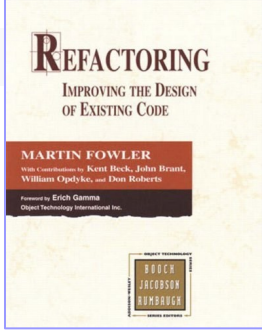
Making code more general in case it's needed later

RATIONALE / ADVICE

- Unused hooks and special cases make code more difficult to understand
- Write code to solve today's problems, and worry about tomorrow's problems when they actually materialize!

WARNING:

Duplicate Code (no generality techniques applied) and ***Speculative Generality*** (generality techniques used abundantly, even when not used) are two opposite extreme practices. Both are bad!



Long Parameter List

Many parameters passed into a method

RATIONALE / ADVICE

- The more parameters a method has, the more complex it is
- Limit the number of parameters you need in a given method, or use an object to combine the parameters

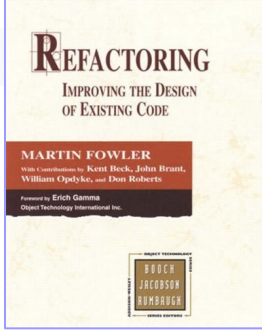
Is this a short parameter list?

```
void foo(String... args) {
    for (String arg : args) {
        System.out.println(arg);
    }
}
```



Data Clumps

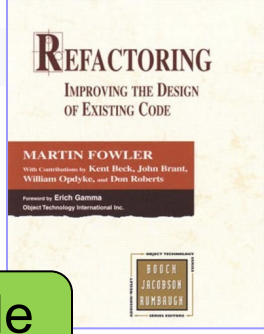
Sets of variables usually passed together in multiple places



RATIONALE / ADVICE

- Bunches of data that hang around together really ought to be made into their own object
- A good test is to consider deleting one of the data values
  - if you did this, would the others make any sense?
  - If they don't, it's a sure sign that you have an object that's dying to be born …

This code smell often occurs simultaneously with the "**Long Parameter List**" (see before)



Primitive Obsession

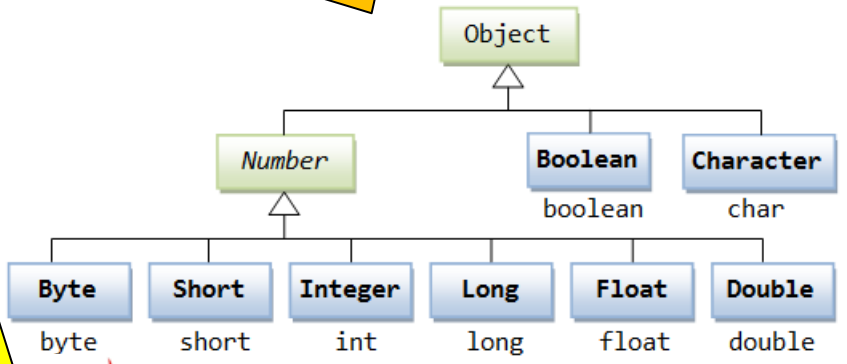
This smell manifests itself when code relies too much on primitive types

RATIONALE / ADVICE

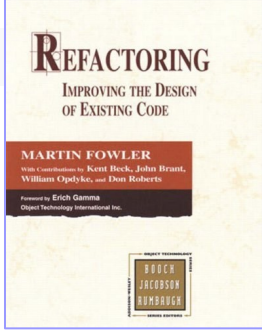
- Primitive types do not allow:
  - adding operations
  - polymorphism (only dangerous casting)

Java API **wrapper classes** provide a mechanism to "wrap" primitive values in an object so that the primitives can be included in activities reserved for objects, like as being added to Collections, or returned from a method with an object return value. Java provides automatic translation between primitive types and wrapper classes (**autoboxing**) and the other way round (**unboxing**).

Java **wrapper classes** hierarchy and corresponding primitive types







Comments (aka Deodorant)

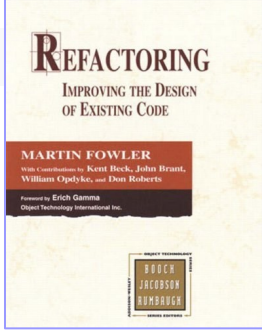
RATIONALE / ADVICE

- Often used as deodorant for other smells
- Not necessarily bad, but may indicate areas where the code is not as clear as it could be
- When you feel like writing a comment, first try to refactor so that the comment becomes superfluous

WARNING:

We are discussing here comments within a method's body! Not method headers where arguments are explained! **Javadoc** is very useful ☺





Temporary Field

Instance variables set only in certain circumstances or fields used to hold intermediate results

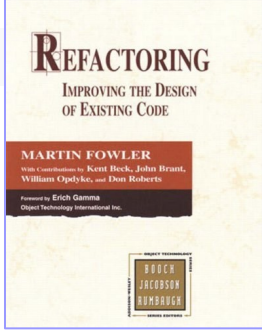
RATIONALE / ADVICE

- Objects sometimes contain fields that don't seem to be needed all the time
- The rest of the time, the field is empty or contains irrelevant data, which is difficult to understand

WARNING:

In weakly type checked languages (e.g. Matlab / Octave) things can be even worst, since the same temp variable can be reused for holding values of different data types.



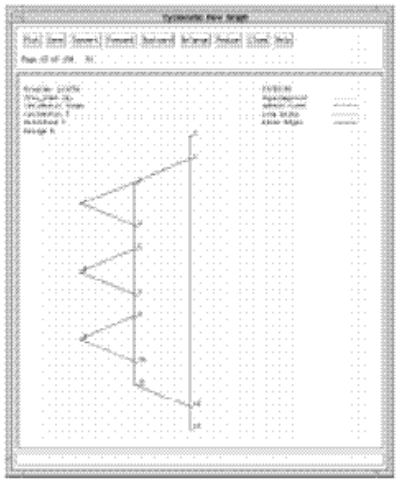


Switch Statements

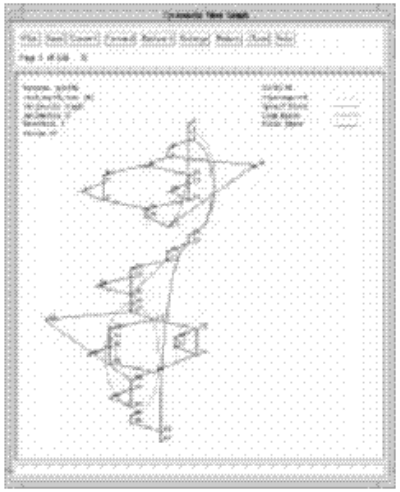
Using long switch statements

RATIONALE / ADVICE

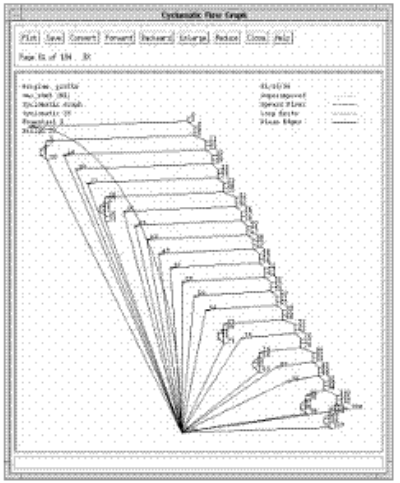
- Switch statements increase code complexity
- Switch statements update requires recompilation
- Use polymorphism instead, whenever possible



Complexity=5



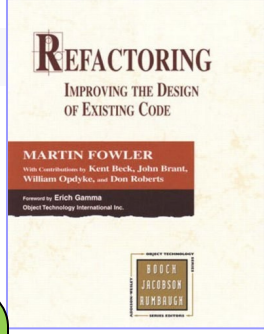
Complexity=17



Complexity=28



Large Class (aka God Class)



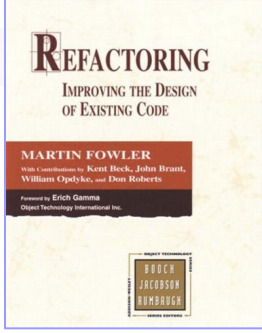
A class with too many instance variables or too much code. In other words, it has excessive responsibility

RATIONALE / ADVICE

- Large classes, like long methods, are difficult to read, understand, and troubleshoot
  - Does the class contain too many responsibilities?
  - Can the large class be restructured or broken into smaller classes?

Data Class

A class whose only purpose is holding data. Only has instance variables, getters, and setters

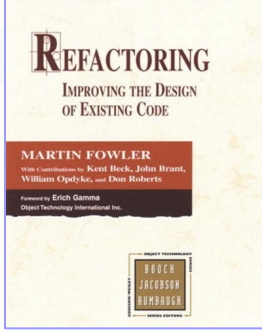


RATIONALE / ADVICE

- Such classes are dumb data holders and are almost certainly being manipulated in far too much detail by other classes
- Sometimes a Data Class was the result of an automatic code generation process (often
  - Automatic programing is good, but other methods (developed by humans) are usually required to operate on the class's data

Lazy Class

A class that isn't doing enough work to justify its maintenance



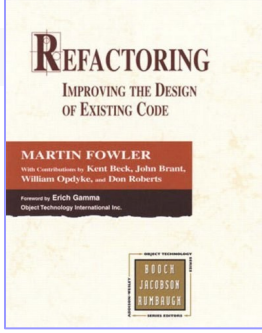
RATIONALE / ADVICE

- Each class you create costs money to understand and maintain
- A class that isn't doing enough to pay for itself should be eliminated

WARNING:

Large / God Class (class with too much responsibility) and **Lazy Class** (class with little responsibility) are two opposite extreme situations. We can sort classes according to responsibility and check if something is wrong with the largest classes and the laziest classes.

Shotgun Surgery

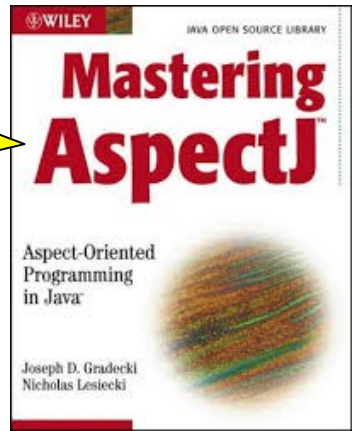


Making one change requires changing code in multiple places

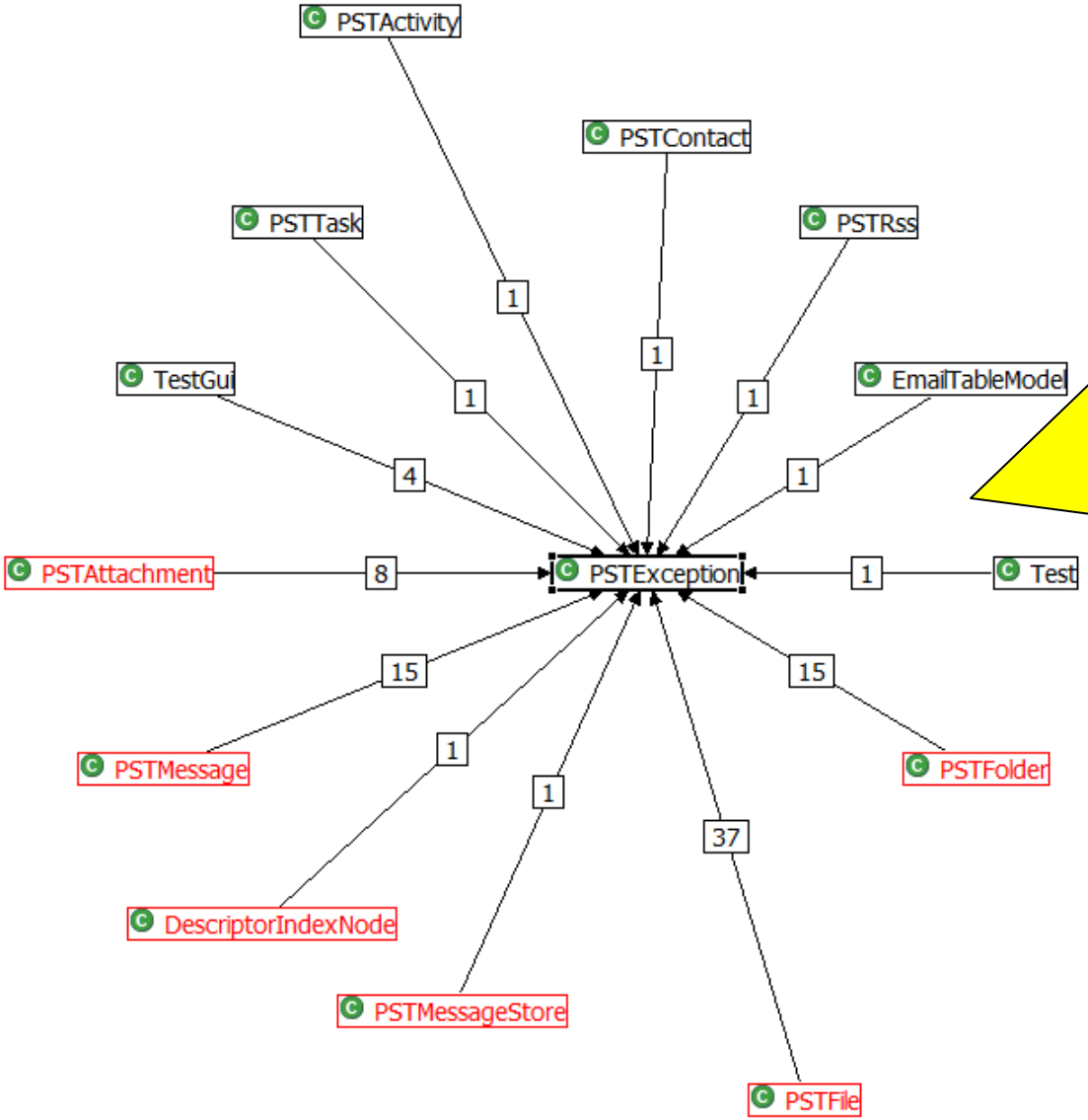
RATIONALE / ADVICE

- If a change in one class requires cascading changes in several related classes, consider refactoring so that the changes are limited to a single class
- This code smell expresses the "**code scattering**" problem (same concerns spread over several classes)

Aspect-Oriented Programing (AOP) is a programing paradigm that aims to increase modularity by allowing the separation of cross-cutting concerns. The most well-known AOP language is AspectJ, that extends Java with aspects!

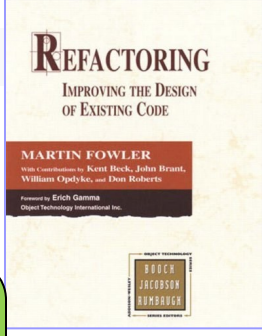


Shotgun Surgery



Changes in the class ***PSTException*** may require changes in the represented classes, since they depend on the former.

The class dependencies diagram offered by the ***CodePro*** tool allows understanding the intensity of this code smell.

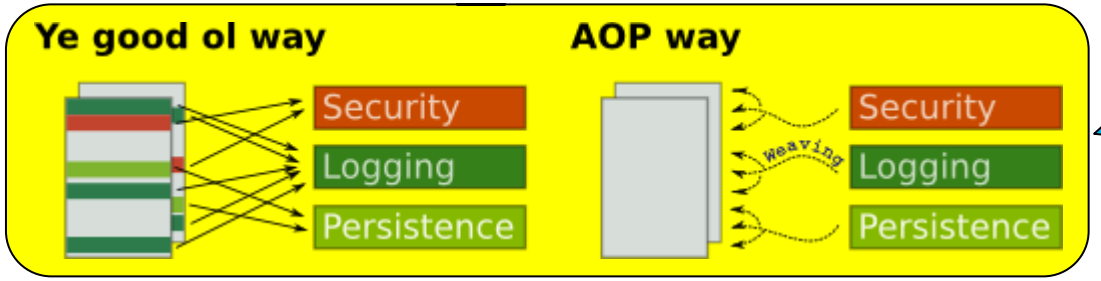


Divergent Change

Occurs when one class is commonly changed in different ways for different reasons. E.g. “three methods change for a new database, while four other methods change for a new financial instrument”.

RATIONALE / ADVICE

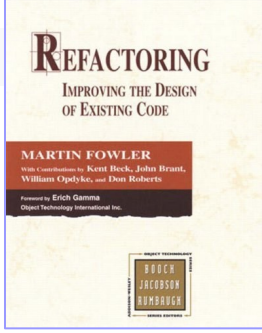
- Separating these divergent responsibilities (aka separation of concerns) decreases the chance that one change affects another and lower maintenance costs
- This code smell denotes what is also called by “**code tangling**” (mixed concerns in the same class)



OOP is the “good old way” ☺
AOP languages aim to solve the scattering and tangling problems

Middle Man

One class simply delegates many of its requests to another class



RATIONALE / ADVICE

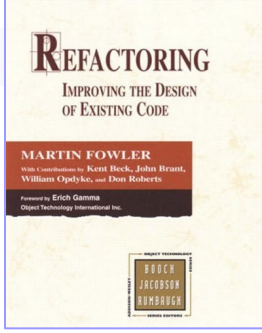
- If a class is delegating all its work, why does it exist?
- Beware classes that are merely wrappers over other classes or existing functionality

WARNING:

Sometimes a **Middle Man** can play a very important role, namely to decouple software components (Adapter **design pattern**) or to provide a simplified API ("Facade" **design pattern**)

Example of Middle Man:

```
class A
{
    B b;
    Object getValue()
    {
        return b.getValue();
    }
}
```



Message Chains

One object asks another object for something, which causes the asked object to ask another object, and so on

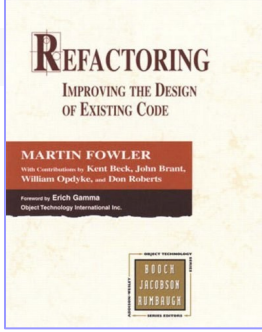
RATIONALE / ADVICE

- Watch out for long sequences of method calls or temporary variables to get routine data
- Intermediaries are dependencies in disguise

Example of Message Chain:

a.getB().getC().getValue();

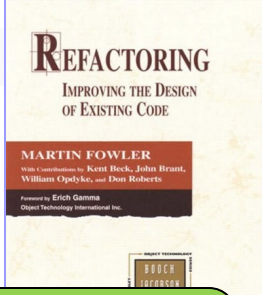
Alternative Classes with Different Interfaces



Occurs when the interfaces of two classes are different on the outside and yet the classes are quite similar on the inside

RATIONALE / ADVICE

- If you can find the similarities between the two classes, you can often refactor to make them share a common interface



Parallel Inheritance Hierarchies

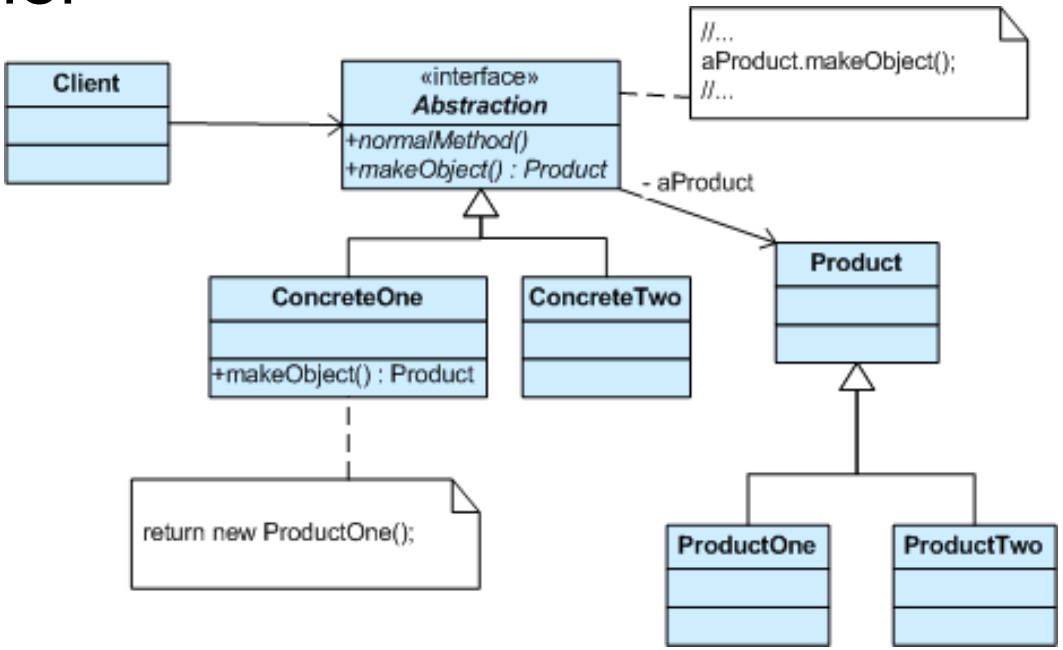
Every time you make a subclass of one class, you have to make a subclass of another

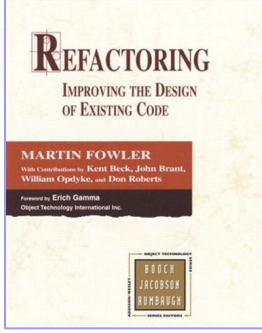
RATIONALE / ADVICE

- This is really a special case of Shotgun Surgery – every time you make a subclass of one class, you have to make a subclass of another

WARNING:

The Abstract Factory design pattern and the Factory Method design pattern use parallel hierarchies. So sometimes those hierarchies may be adequate!





Incomplete Library Class

Libraries that don't provide all needed methods

RATIONALE / ADVICE

- Occurs when responsibilities emerge in our code that clearly should be moved to a library class, but we are unable or unwilling to modify the library class to accept these new responsibilities



- **Overview**: is one of the most popular static analysis tools, capable of identifying a wide range of code smells, bugs, and security vulnerabilities.
- **Coverage of Fowler's catalog**: covers a broad spectrum of Fowler's code smells, such as *Long Method, Large Class, Feature Envy,* and *Duplicate Code*. It also allows customization and extension through rules to capture other smells.
- **Strengths**: Wide language support (Java, C#, JavaScript, Python, etc.). Integrates well with CI/CD pipelines. Excellent reporting and visualization features.
- **Weaknesses**: Some rules may require fine-tuning for specific project setups.The free version has limited functionality compared to the commercial version.



- **Overview:** A static analysis tool specialized in detecting design and architectural smells in object-oriented code, with metrics and visualization support.
- **Coverage of Fowler's catalog:** Detects many Fowler-inspired smells such as *Long Method*, *Large Class*, *Feature Envy*, *Data Class*, *Shotgun Surgery*, and *Divergent Change*.
- **Strengths:** High alignment with Fowler's code smells and OO design principles. Offers rich metrics and visualizations (e.g., dependency graphs, smell summaries). Supports multiple languages (Java, C#, Python).
- **Weaknesses:** Commercial tool (limited free features). Focused on design-level smells — not security or performance analysis. Less seamless integration with CI/CD compared to SonarQube.



Codacy



- **Overview**: Codacy is a cloud-based code quality tool that supports many programming languages and helps detect code smells, bugs, and security issues.
- **Coverage of Fowler's Code Smells**: Codacy has rules that cover many of the smells in Fowler's catalog such as *Long Method, God Object, Duplicated Code, Shotgun Surgery,* and *Unnecessary Object Creation.*
- **Strengths**: Supports many languages and integrates easily with version control systems (e.g., GitHub, GitLab). Automated code review with smart suggestions for refactoring.
- **Weaknesses**: Some functionality is behind a paywall. It may generate false positives depending on how the rules are configured.



JDeodorant



- **Overview:** A specialized Eclipse plug-in focused on detecting and refactoring design flaws in object-oriented software. It identifies code smells and provides automated refactoring suggestions.
- **Coverage of Fowler's catalog:** Detects key design smells such as *Long Method*, *Large Class (God Class)*, *Feature Envy*, and *Type Checking (Switch Statements)*. Focused on design-level issues rather than general code quality or security.
- **Strengths:** Deep design analysis with automated refactoring support. Tight Eclipse integration. Research-based detection heuristics.
- **Weaknesses:** Limited to Java. Narrower scope than general-purpose tools. No native CI/CD integration.